

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C02: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I SOWMIYA. D (192511063) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

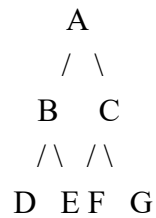
Signature of the student: Sowmiya. D

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

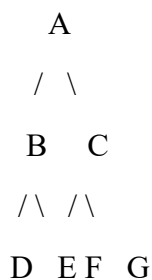
Iteration Queue Visited Node

Iteration	Queue	Visited Node
1		2
3		
4		
5		
6		
7		

Questions:

- Write the BFS traversal order.
- Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

3. Initial State :

1 3 6
5 0 2

4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached. Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

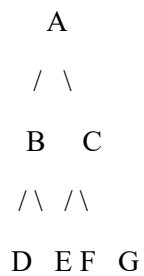
Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Question 1: Breadth-First Search (BFS)

Aim

To perform **Breadth-First Search (BFS)** on the given graph starting from node **A** and determine the traversal order and queue contents at each iteration.

Given Graph



Algorithm (BFS)

1. Start from the root node.
2. Mark the node as visited.
3. Insert the node into a queue.
4. Remove the front node from the queue.
5. Visit all its unvisited adjacent nodes.
6. Insert the adjacent nodes into the queue.
7. Repeat until the queue becomes empty.

Dry Run Table

Iteration Visited Node Queue After Visiting

1	A	B, C
2	B	C, D, E
3	C	D, E, F, G
4	D	E, F, G

5 E F, G

6 F G

Iteration Visited Node Queue After Visiting

7 G Empty

Explanation

Iteration 1

- Visit **A**.
- Enqueue its children **B** and **C**.
- Queue = **B, C**

Iteration 2

- Dequeue **B** and visit it.
- Enqueue **D** and **E**.
- Queue = **C, D, E**

Iteration 3

- Dequeue **C** and visit it.
- Enqueue **F** and **G**.
- Queue = **D, E, F, G**

Iteration 4

- Visit **D**.
- No children.
- Queue = **E, F, G**

Iteration 5

- Visit **E**.
- No children.
- Queue = **F, G**

Iteration 6

- Visit **F**.
- No children.
- Queue = **G**

Iteration 7

- Visit **G**.
- Queue becomes empty.
- BFS terminates.

Traversal Order

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Advantages of BFS

- Finds the shortest path in an unweighted graph.
- Visits nodes level by level.
- Simple to implement using a queue.
- Guarantees visiting every reachable node.

Applications

- Shortest path finding.
- Social networking.
- GPS navigation.
- Web crawling.
- Network broadcasting.

Result

Thus, the **Breadth-First Search traversal** of the given graph is

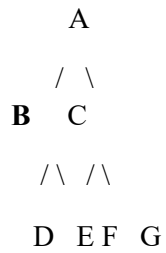
$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Question 2: Depth-First Search (DFS)

Aim

To perform **Depth-First Search (DFS)** on the given graph starting from node **A** and determine the traversal order and stack contents.

Given Graph



Algorithm (DFS)

1. Start from the root node.
2. Visit the node and mark it as visited.
3. Push the node onto the stack.
4. Visit the leftmost unvisited child.
5. Continue until a leaf node is reached.
6. Backtrack using the stack.
7. Repeat until all nodes are visited.

Dry Run Table

Iteration Visited Node Stack After Visiting

1	A	C, B
2	B	C, E, D
3	D	C, E
4	E	C
5	C	G, F
6	F	G
7	G	Empty

Explanation

Iteration 1

- Visit **A**.
- Push **C** then **B** onto the stack (so B is processed first).

- Stack = C, B

Iteration 2

- Pop B and visit it.
- Push E then D.
- Stack = C, E, D

Iteration 3 • Pop D

and visit it.

- No children.
- Stack = C, E

Iteration 4 • Pop E

and visit it.

- No children.
- Stack = C

Iteration 5

- Pop C and visit it.
- Push G then F.
- Stack = G, F

Iteration 6

- Pop F and visit it.
- Stack = G

Iteration 7

- Pop G and visit it.
- Stack becomes empty.
- DFS terminates.

Traversal Order

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

Advantages of DFS

- Uses less memory compared to BFS.
- Suitable for deep graph traversal.

- Useful for path finding and cycle detection.
- Easy to implement using recursion or stack.

Applications

- Topological sorting.
- Maze solving.
- Cycle detection.
- Backtracking algorithms.
- Connected component detection.

Result

Thus, the **Depth-First Search traversal** of the given graph is

A → B → D → E → C → F → G

3. Breadth-First Search (BFS) for 8-Puzzle

Initial State

1 3 6
5 0 2
4 7 8

Goal State

1 2 3
4 5 6
7 8 0

Algorithm: Breadth-First Search (BFS)

Step 1: Place the initial state into the queue.

Step 2: Remove the first state from the queue.

Step 3: Check whether it is the goal state.

Step 4: If not, generate all possible child states by moving the blank (0) in all valid directions.

Step 5: Add all unvisited child states to the end of the queue. **Step**

6: Repeat the process until the goal state is found.

Trace of BFS

Iteration 1

Current State

1 3 6
5 0 2
4 7 8

Blank is in the center.

Possible moves:

- Up

- Down
- Left
- Right

Generated states:

Up

1 0 6

5 3 2

4 7 8

Down

1 3 6

5 7 2

4 0 8

Left

1 3 6

0 5 2

4 7 8

Right

1 3 6

5 2 0

4 7 8

Queue After Expansion:

[

Up,

Down,

Left,

Right

]

Iteration 2

Current State

1 0 6

5 3 2

4 7 8

Possible moves

- Left
- Right
- Down

Generated States

0 1 6

5 3 2

4 7 8

1 6 0

5 3 2

4 7 8

1 3 6

5 0 2

4 7 8

(The last state is already visited and is ignored.)

Queue After Expansion

[
Down,
Left,
Right,
0 1 6, 1 6 0
]

Iteration 3

Current State

1 3 6

5 7 2

4 0 8

Generated States

1 3 6

5 0 2

4 7 8

(Visited)

1 3 6

5 7 2

0 4 8

1 3 6

5 7 2

4 8 0

Queue After Expansion

[
Left,
Right,
0 1 6,
1 6 0,
5 7 2
0 4 8,
5 7 2
4 8 0
]

Iteration 4

Current State

1 3 6

0 5 2

4 7 8

Generated States

0 3 6

1 5 2

4 7 8

1 3 6

4 5 2

0 7 8

(Other generated state is already visited.) Queue is updated with new states.

Iteration 5

Current State

1 3 6

5 2 0

4 7 8

Generated States

1 3 0

5 2 6

4 7 8

1 3 6

5 2 8

4 7 0

Visited state is ignored.

Queue is updated.

Table

Iteration Current State Queue Before Expansion Queue After Expansion

1	Initial State	[Initial]	[Up, Down, Left, Right]
2	Up State	[Up, Down, Left, Right]	[Down, Left, Right, New States]
3	Down State	[Down, Left, Right,...]	[Left, Right, New States]
4	Left State	[Left, Right,...]	[Right, New States]
5	Right State	[Right,...]	[New States Added]

Solution Path

BFS continues expanding level by level until it reaches

1 2 3

4 5 6

7 8 0

The solution path is obtained by tracing back from the goal node to the initial node using parent pointers. Since BFS explores all states level by level, the first goal reached is guaranteed to be the shortest path.

Answers to the Questions

1. Which node is expanded in each iteration?

- Iteration 1 – Initial State
- Iteration 2 – Up State
- Iteration 3 – Down State
- Iteration 4 – Left State
- Iteration 5 – Right State

BFS always expands nodes in **First-In, First-Out (FIFO)** order.

2. How many states are generated in total?

- Initial state = **1**
- First expansion = **4 states**

- Subsequent expansions generate additional child states while excluding already visited states. Thus, BFS generates **many intermediate states until the goal is reached**. The exact total depends on when the goal is encountered during the search.

3. Write the complete solution path from the initial state to the goal state.

Initial State



Intermediate States



More Intermediate States



Goal State

1 2 3

4 5 6

7 8 0

The shortest path is obtained automatically by BFS because it explores all nodes level by level.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle?

Breadth-First Search explores nodes **level by level**. Every move in the 8-puzzle has the same cost (1 move). Therefore, BFS reaches the goal using the **minimum number of moves** before exploring deeper levels. This guarantees that the first solution found is the **shortest and optimal solution**.